real-mb-fidelity___fidelity-binomial-f

An AgentMPI run analysis

Abstract

This is the shallow cell of the reduction-fidelity experiment: eight ranks, four uniquely identified facts each, folded to rank 0 by a model-executed merge arranged as a binomial tree of depth 3. No rank was erased. Retention is 1.000, all 32 identifiers reach the root, none is fabricated, and this is not merely the aggregate figure — I recovered all seven intermediate accumulators from this run's `spool/` directory under `runs/real-mb-fidelity/` and every one of them carries exactly four identifiers per leaf beneath it. The sibling runs `-chain-f` (depth 7) and `-flat-f` (depth 7) score the same 1.000. The tree-shape hypothesis — that a deeper fold loses more, because each fold is a lossy model call — therefore finds no support here, but the more important point is that this configuration could not have tested it: the merges peaked at 622 output tokens against a nominal 900-token budget, and the contract actually recorded in the fabric carries `max_tokens: null`, so the budget was an advisory sentence in a prompt and not a constraint at all. An operator that is never forced to discard is lossless at any depth, and the depth-dependent loss the experiment was built to find lives in the enforced, incompressible sibling campaign `inc-mb-fidelity-inc`, where retention is 0.385 identically at depths 2, 3, 7 and 7. What this run does establish, cleanly, is the cost side: three of its seven operator applications lie on the critical path instead of seven, and it finishes in 58.25 s for $0.0439 against the chain's 130.57 s and $0.0586 and the flat reduction's 152.60 s and $0.0594 — at identical quality.

1 What this run is

property	value
run	`real-mb-fidelity__fidelity-binomial-f`
experiment	–
campaign label	–
declared world size	8
ranks appearing in the log	8
executors	`broker` × 8, `worker` × 31
events	100
wall time	338.46 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.31×	total busy time over wall time
parallel efficiency	3.9%	achieved parallelism per rank
idle fraction	96.1%	rank-seconds paid for and unused
peak ranks busy	4	of 8 in the world
serial fraction of active time	48.4%	time with exactly one rank busy
load imbalance	1.73×	slowest rank's busy time over the mean
coordination share	4.7%	rank-seconds blocked in collectives, of those available
coordination in flight	17.2%	wall clock with any rank inside a collective
rank reattachments	31	fresh agent processes that took over a rank role
agent calls	7	invocations that reached a model
agent latency p50 / p95	16.51 / 20.51 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	1,483	0 deferred under rendezvous
tokens in / out	3,747 / 2,180	prompt and completion
cost	$0.0439	0.0202 $/kTok out

3 What the analysis notices

- [note] Parallel efficiency is 3.9%: 96.1% of the rank-seconds paid for went unused.
- [note] Load imbalance is 1.73×: the slowest rank was busy far longer than the mean.
- [ok] 31 rank reattachment(s) occurred, up to incarnation 12: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	46.04	14.4	3	3	0	3	0	0.0	finalized
1	0.00	0.0	0	0	1	0	153	0.0	finalized
2	13.58	9.0	1	1	1	1	231	0.0	finalized
3	0.00	0.0	0	0	1	0	153	0.0	finalized
4	32.17	28.5	2	2	1	2	366	0.0	finalized
5	0.00	0.0	0	0	1	0	153	0.0	finalized
6	14.61	4.4	1	1	1	1	274	0.0	finalized
7	0.00	0.0	0	0	1	0	153	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

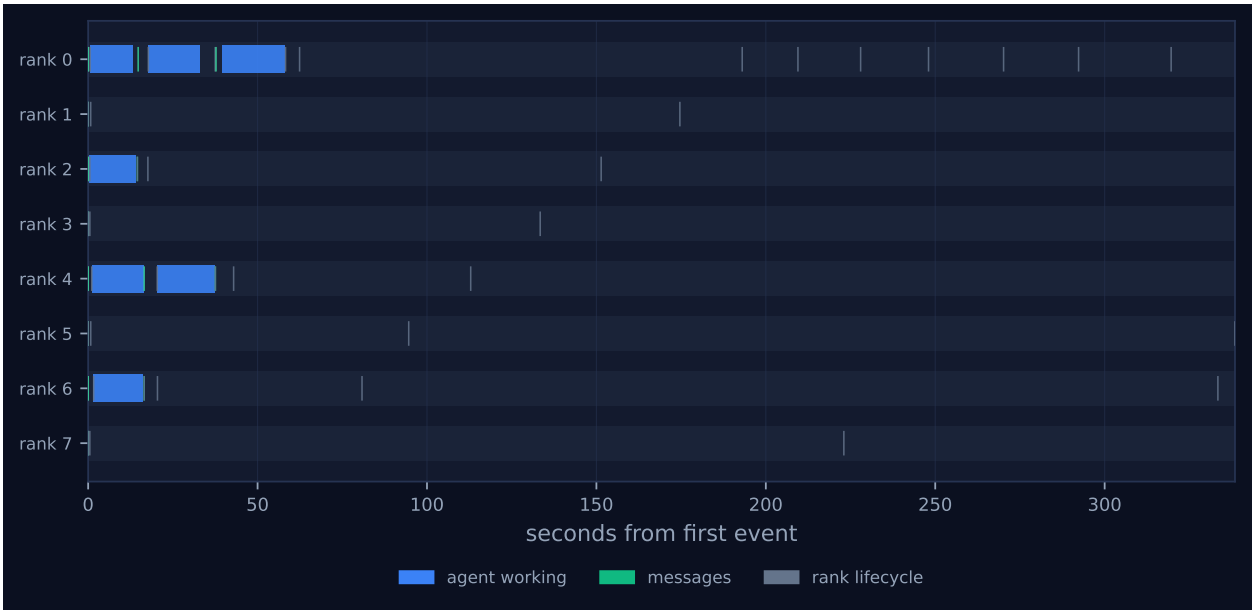

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

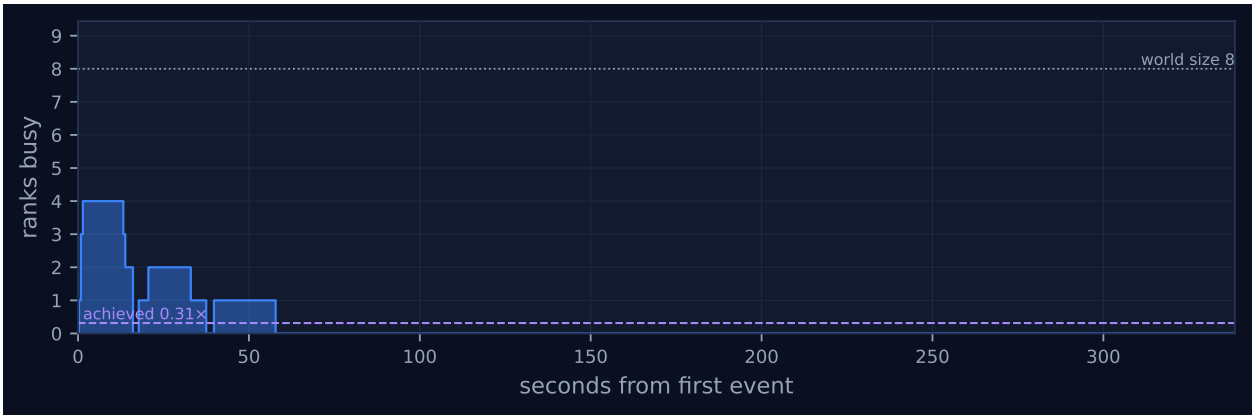

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

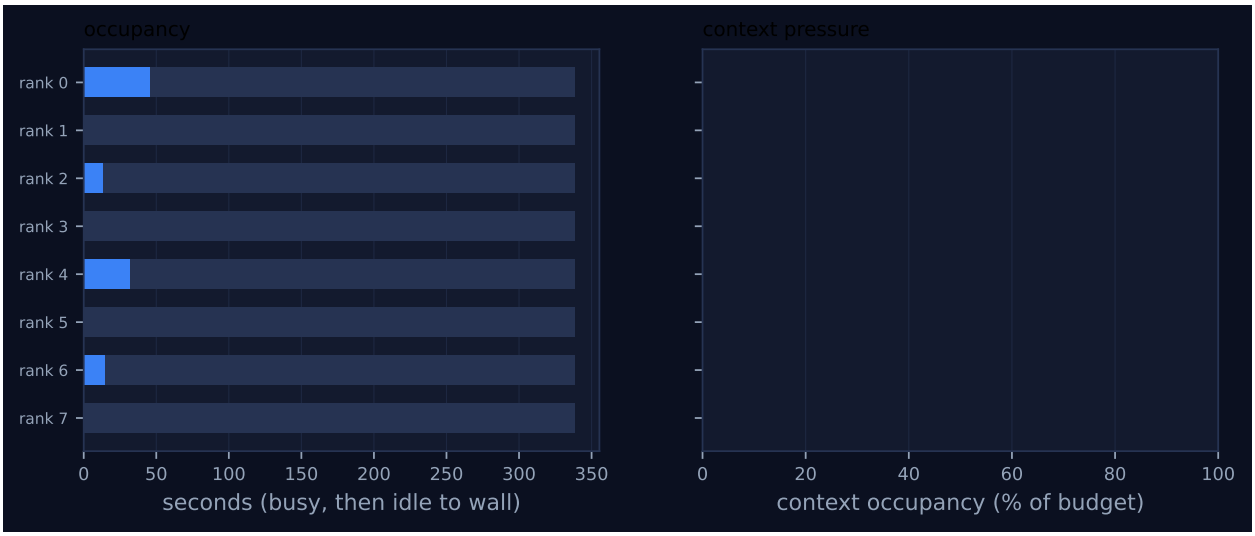

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	binomial	–	8	8	3	3	7	7	1,392	58.25	58.25	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 and the dashboard screenshot show a descending staircase of concurrency in the first minute and then nothing. Four lanes — ranks 0, 2, 4 and 6 — open with a bar at $t \approx 0.5$–1.6 s; two of them (0 and 4) carry a second bar from about 18 s; rank 0 alone carries a third from 39.5 s to 58.0 s. Ranks 1, 3, 5 and 7 have no bar anywhere. That is the binomial fan-in drawn literally: at $p = 8$ the odd virtual ranks send their leaf report to a partner and leave, which they do at $t = 0.01$–0.02 s, and `coll.reduce` records `fold_depth: 0` for each of them because they applied the operator zero times. The four even ranks perform the level-1 merges concurrently, ranks 0 and 4 the level-2 merges, and rank 0 the level-3 merge. Peak ranks busy is 4 of 8, mean concurrency when active is 1.98, and 48.4% of the active time has exactly one rank busy — the last two levels.

The gaps in rank 0's lane are the only waits in the run and both are short. Rank 0 finished its level-1 merge at 13.03 s but could not start level 2 until rank 2's level-1 result arrived at 14.76 s: a 1.73 s wait. It finished level 2 at 32.81 s and waited until 37.75 s for rank 4's level-2 result, which itself depended on rank 6's level-1 merge completing at 16.24 s: a 4.48 s wait. So of the 58.25 s collective span, rank 0 held a task for 46.04 s and waited 6.2 s on subtrees, and the residue is broker

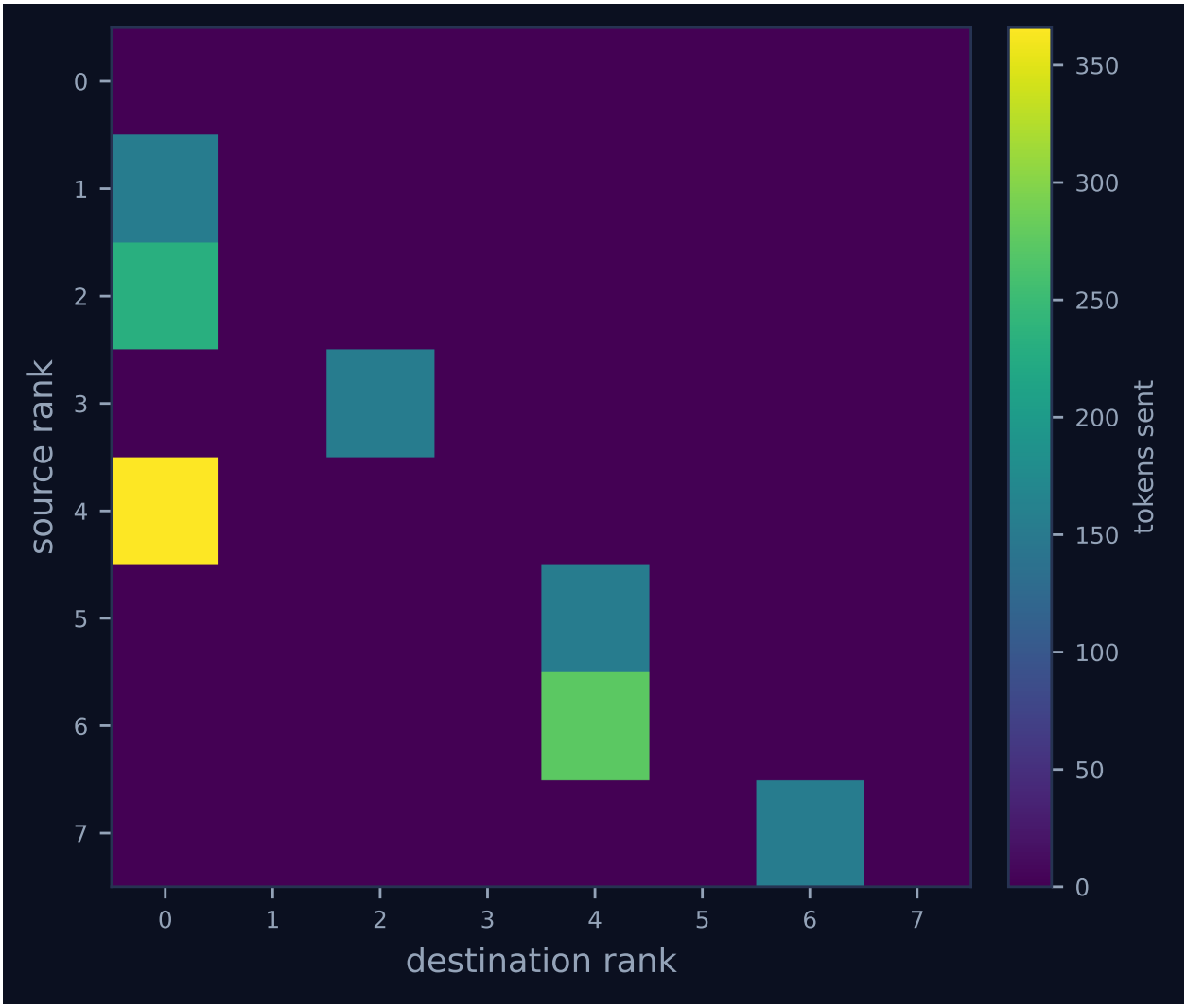

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

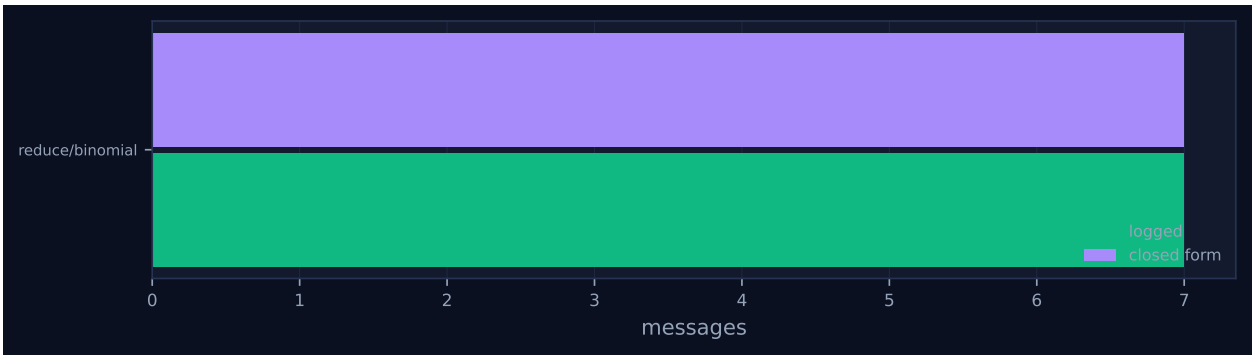

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

turnaround. That is what a well-shaped tree looks like — the root is the busiest rank because it sits on every level, and its idleness is bounded by the skew between sibling subtrees rather than by a serial predecessor chain. Compare the chain sibling, whose timeline is one bar at a time in strict rank order with no overlap at all, and the flat sibling, whose rank-0 lane is a single unbroken train of seven bars while the other seven lanes are empty.

The communication matrix, Figure 4, is the tree's other signature and shows the accumulation directly. Seven edges: $3 \to 2$, $5 \to 4$, $7 \to 6$ at 153 tokens each, $6 \to 4$ at 274, $2 \to 0$ at 231, and $4 \to 0$ at 366. Weight is a proxy for subtree size — a leaf report costs 153 tokens on the wire, a two-rank accumulator 231–274, a four-rank accumulator 366 — and the three edges converging on rank 0 carry 750 of the run's 1,483 message tokens. The 153-token leaf, incidentally, is 12 tokens more than the 141 the flat sibling sends for the same payload, because the binomial branch in `algorithms.py` wraps the value in an envelope carrying `acc`, `depth`, `weight` and `vr` while `flat` sends the bare report.

The last thing the timeline shows is an artefact, and it is large enough to distort the headline table. `job.finish` is emitted at 58.26 s, but the log runs to 338.46 s: the final 17 of 100 events are `rank.init` records for worker processes re-attaching to rank roles after the job has completed, rank 0 reaching incarnation 12. Those grey ticks are the sparse right-hand two-thirds of every lane in Figure 1, and 82.8% of the reported wall time is that tail. I return to what it does to the metrics in the next section.

7 Interpretation

Most of the run's shape is true by construction. The world size, the four items per rank, the merge prompt, the 900-token nominal budget and the choice of the binomial algorithm are all harness decisions in `bench_fidelity`; the fold structure is fixed by `algorithms.py`, and both checkable predictions hold — 7 messages against a closed form of 7, 3 rounds against 3, so `model_checks_agree` is 1 of 1 and Figure 5 shows no red diamond. Worth naming, because the hypothesis is usually stated in terms that this run's siblings contradict, is that the depth contrast in this campaign is only two-valued. The `flat` branch of `algorithms.py` sorts the contributions by source rank and then applies the *binary* operator $p - 1$ times in sequence at the root, setting `fold_depth = p - 1 = 7` with `rounds = 1`; the trace of `-flat-f` duly shows seven successive `merge:d1`…`merge:d7` calls on rank 0. So this campaign compares depth 3 against depth 7 twice, not 3 against 7 against 1. The single-application-of-p-inputs form is the variadic kernel used only by `kary`, which this campaign did not run; it appears in the `sat-mb-fidelity` and `inc-mb-fidelity-inc` siblings at $k = 4$, depth 2.

The retention result is what emerged, and it refutes the prediction it was built to confirm. Retention is 1.000 in this run, 1.000 in `-chain-f` and 1.000 in `-flat-f`, with `n_hallucinated_ids` zero in all three, so a fold depth of 3 and a fold depth of 7 are indistinguishable on quality. Because an aggregate retention figure can hide a reduction that keeps a complete prefix and erases the rest, I checked the fold item by item rather than trusting the aggregate. The seven accumulators in the spool carry identifier sets $\{4,5\}$, $\{2,3\}$, $\{6,7\}$, $\{0,1\}$, $\{0,1,2,3\}$, $\{4,5,6,7\}$ and $\{0,\ldots,7\}$ — exactly the binomial tree's fold order — with 8, 8, 8, 8, 16, 16 and 32 identifiers respectively, which is four per leaf at every node. Nothing was dropped at any intermediate level and then concealed by a later re-expansion; per-rank retention is genuinely 1.000 for all eight ranks. The erasure failure mode is real, but it belongs to the enforced campaign: in `inc-mb-fidelity-inc`, ranks 0–2 are retained completely, rank 3 keeps 1 of 12 items and ranks 4–7 keep none, with a rank-position correlation of -0.86 — and that pattern is byte-for-byte the same at k-ary depth 2, binomial depth

3, and chain and flat depth 7. Depth does not matter there either. It is the enforced budget that sets what survives.

The reason nothing was lost here is arithmetic, and it is worth quantifying because it tells a harness author when to care. Reading the accumulator sizes off the flat sibling's left fold, which grows one rank at a time, output tokens track $18.3 + 85.2 \times$ (ranks folded) to within one token over $n = 2 \ldots 7$. At $p = 8$ that extrapolates to 700 tokens and the 900-token budget is first reached at $p \approx 10.3$, about 41 items. Eight ranks contributing four short items each is simply below the saturation threshold, so the operator had slack and used it: the observed 32-item result is 622 tokens, 69% of the budget and 78 tokens under the extrapolation, because the final merge re-encoded into a terser register. At 19.4 tokens per item the budget's capacity is roughly 46 items. The paper's own reading is the right one — fold depth penalises fidelity only when the reduction is capacity-bound — and this cell is the unsaturated control that establishes the antecedent, not a test of the consequent.

There is a second and stronger reason the retention number cannot bear the weight the hypothesis wanted to put on it: in this run the budget was never enforced. The `agent_calls` row for every one of the seven merges records `contract` = `{"name": "MergedReport", ..., "max_tokens": null, "semantics": ""}` while `meta` records `{"max_tokens": 900}`. The budget went to the executor as a generation hint, which a worker is free to ignore, and never to the contract checker that would have rejected an overrun and retried. All seven calls show `attempt: 1` and the run reports zero contract violations, which under a null bound is uninformative rather than reassuring. The `inc-mb-fidelity-inc` fabrics show the corrected regime for contrast — `max_tokens: 450` with non-empty semantics, and its `flat` cell contains eight calls rather than seven: `merge:d3` returned 472 tokens on attempt 1, was rejected for overrunning the 450-token bound, and was re-issued as attempt 2, which returned 437. The `sat-mb-fidelity` campaign, still advisory but with a 450-token budget and twelve items per rank, is where the consequence is visible: its merges emitted up to 698 tokens, 55% over the stated limit, and still scored retention 1.000. This run did not overrun only because 32 short items fit comfortably under 900. Both facts are needed to read it: the budget was not enforced, and it made no difference here because nothing came close to it.

What the run does establish is the cost axis at held-constant quality, and here the tree wins outright. All three algorithms perform exactly seven operator applications and seven messages; they differ only in how many applications lie on the critical path. Pooling all 21 merges across the three runs, per-call latency fits $12.31\,\text{s} + \text{output\_tokens}/57.0\,\text{tok/s}$ with $r = 0.789$, so a merge costs about 12 s of fixed overhead plus generation. Three levels at this run's median 16.51 s predict 49.5 s against a measured 58.25 s, the difference being the subtree waits above; the chain's seven levels at its median 18.51 s predict 129.6 s against 130.57 s measured, and the flat reduction's seven serial applications at its median 20.51 s predict 143.6 s against 152.60 s. The tree is therefore 2.24× faster than the chain and 2.62× faster than the flat reduction, and it is also the cheapest: 3,747 prompt and 2,180 completion tokens for $0.0439, against $0.0586 and $0.0594, a 25–26% saving, because a shallow tree never carries a large accumulator through many prompts. Note that flat is the *slowest* despite needing a single communication round, which is the cost model's prediction confirmed rather than an anomaly.

One number the metrics do not compute is worth adding, because it strengthens the same conclusion by a mechanism the closed form does not model. Decomposing each call into `broker.enqueue` → `broker.claim` → `broker.complete`, this run spent 12.27 s of its 118.67 s of aggregate call latency waiting for a worker to claim a pending call (10.3%), against 3.20 s of 128.79 s in the chain (2.5%) and 26.22 s of 150.93 s in the flat reduction (17.4%). The flat root paid roughly 3.7 s of worker attach seven times over, because rank 0 reached incarnation 8 for seven calls — one fresh worker

process per application. A tree amortises that cost across concurrent branches; a serial fold at one rank pays it in full every time.

Two of the four flagged findings are artefacts of the same defect and should not be read as properties of the run. "Parallel efficiency is 3.9%" and "load imbalance is 1.73×" both divide by `wall_s`, which `analysis.py` defines as last event minus first event, and 82.8% of that span is the post-completion worker churn described above. Recomputed against the 58.25 s collective span, achieved parallelism is 1.83× rather than 0.31× and the idle fraction is 77.2% rather than 96.1%. This matters beyond one cell, because the published figures rank the three algorithms 0.31× (binomial), 0.45× (chain), 0.82× (flat) — the exact reverse of the truth, since binomial is the only one of the three that achieved any concurrency at all (peak ranks busy 4, against 1 and 1). Corrected, the ordering is 1.83×, 0.96×, 0.82×. I regard this as a genuine defect in the derived metrics rather than a property of the protocol, and it is worth fixing at source: bounding the analysis window at `job.finish`, or excluding lifecycle-only events from the span, would repair every run whose worker pool outlives its job. The remaining two findings are correct and expected: the 31 reattachments are the broker's normal mode of operation, and the cost-model agreement is the check passing.

8 Threats to this reading

The most serious threat is that retention 1.000 may not measure information transport at all. In the compressible configuration `make_fact_report` builds each item as a throughput of $100 + 7 \times \text{rank} + i$ under workload `['nominal', 'degraded', 'saturated'][i % 3]`, so an item's entire payload is a deterministic function of its identifier, and `fact_retention` scores only whether the identifier appears. A merge that had never seen `F-6-2` could emit it correctly by extrapolating the pattern from its neighbours, and would be scored as having retained it. Every claim in this document about what survived is therefore a claim about identifier survival on a guessable payload. This is precisely why the `-inc` variant exists — high-entropy serials and checksums, scored by `value_retention` on the payload as well as the label — and it means the strongest statement this run can support is that a compressing operator protects identifiers, which its own docstring already asserts.

Second, the final result is byte-identical across all three algorithms, and I cannot fully exclude that the worker pool remembered rather than recomputed it. Digest `e1801060bfbe` (1,498 bytes) is the only blob shared by all three runs' content-addressed stores, and three different prompts — 832 tokens here, 838 in the chain, 914 in the flat reduction — produced it. Two four-rank accumulators coincide across runs the same way, this run's $\{0, 1, 2, 3\}$ with the flat run's and its $\{4, 5, 6, 7\}$ with the chain's, again from different prompts. The runs shared one persistent pool of Cursor subagents and ran sequentially, and rank 0 performed the final merge in all three, so the mechanism for reuse existed. The evidence favours convergence, and the discriminating test is the wording register. The population uses a small set of discrete renderings, from the verbatim input at 119 bytes per item down to `[F-0-0] system-0.0:  100 units/s, nominal.` at 46, and a merge inherits its accumulator's register: the flat run holds one register unchanged from its first merge to its sixth (84, 83, 82, 82, 81, 81 bytes per item), the chain adopts a different one at its second merge and holds it (68, 67, 67, 66, 66), and this run's four subtrees independently adopt four different registers (69, 98, 119, 84) which are then inherited upward. The only step in any of the three runs where the register is *not* inherited is the last one, where 32 items must be rendered and all three paths land on the same minimal encoding — and the title is house style rather than coincidence: all 21 merges open their title with `Consolidated report`, and 20 of the 21 use the exact form `Consolidated report:  component groups ...`, the sole exception substituting `from` for the colon. Decisively

against wholesale reuse, this run and the flat run share no intermediate at all with each other beyond those two four-rank nodes, the chain and flat runs share only the final result, and each run's intermediates track its own fold order and nothing else: the flat run's unique seven-rank accumulator (2,330 bytes) exists in no other store. A worker replaying remembered text would not have produced it. Still, byte-identity of a free-form title across three runs is a coincidence I can explain but not rule out.

Third, this is one trial of one campaign, and the three cells differ in more than their tree. The latency ordering is safe: per-call latency across all 21 merges spans 13.01–27.01 s with a standard deviation of 3.56 s, so the 72 s gap between this run and the chain is many times the variability of a single call and is predicted by mechanism — identical application counts, three on the critical path instead of seven. The 22 s gap between the chain and the flat reduction is not safe on one trial each; it is consistent with flat's larger root prompts (up to 914 tokens against the chain's 838) and its sevenfold worker-attach cost, but I cannot separate it from run-to-run variance in model output. Nothing in this campaign licenses a claim about the *distribution* of any of these quantities, and the campaign configuration records `reps: 5` and `trials: 32`, neither of which applies to the fidelity bench — those parameters belong to other benches, and reading them as replication here would be wrong.

Fourth, several accounting figures in the generated tables mean less than they appear to. Every `msg.recv` in this run records `admitted: false` because the collective's internal receives pass `admit=False`, so context occupancy is 0% and context high water 0 on all eight ranks: this run exercises no context pressure whatsoever and says nothing about admission. The `job.finish` payload reports `tokens_deferred: 1483` while `metrics.json` reports 0, because the runtime's counter includes unadmitted receipts and the analysis's counts only rendezvous back-pressure — both are right under their own definitions and a reader comparing them would be misled. The reported `rank_position_correlation` of 0.0 is not a measurement of positional fairness but an undefined quantity: at retention 1.000 the per-rank variance is zero, the correlation's denominator vanishes, and the code returns 0.0 by a guard. And the $0.0439 is modelled, not billed — 3.0/Mtok in and 15.0/Mtok out from the calibration block reproduce it exactly from the token counts, so it is a price card applied to an offline token estimate with `tokenizer_exact: false`, not an invoice.

Finally, the comparison this run anchors within the paper is not the one the paper quotes. `make_tables.py` populates `\NFidTree` and its companions from whichever broker-executed, non-capacity-bound block iterates last, which is `sat-mb-fidelity` rather than this campaign, so the published "at equal quality" figures are 78 s, 1,377 s and 198 s for a 17.8× speedup. This campaign's own numbers are 58 s, 131 s and 153 s for 2.24×. Both blocks are agent-executed and unsaturated and both support the same direction, but the macro carries no label identifying which configuration it came from, and the two differ by nearly an order of magnitude in effect size. Anyone reading 17.8× as a property of $p = 8$ trees in general would be over-reading it.